

12 — Product Evolution Architecture

Purpose

The purpose of this document is to define the evolution strategy of Mosaica.

This module establishes how Mosaica can grow, improve and adapt over time while preserving architectural consistency.

The objective is to ensure that future changes are introduced:

- intentionally,
 - sustainably,
 - without unnecessary complexity,
 - while protecting the original product vision.
-

Product Evolution Scope

This module defines:

- feature evolution principles,
- version strategy,
- change management,
- future expansion approach.

The purpose is not to predict every future feature, but to establish a framework for responsible growth.

Product Evolution Position

Product Evolution connects the current system with future possibilities.

Current Product

↓

Validated Improvements

↓

Future Capabilities

↓

Evolved Product

Evolution Responsibilities

Feature Growth

Responsible for deciding how new capabilities are introduced.

Product Consistency

Responsible for protecting the original Mosaica vision.

Complexity Management

Responsible for preventing unnecessary expansion.

Long-Term Sustainability

Responsible for ensuring future changes remain maintainable.

Product Evolution Principles

Principle 1 — Value Before Expansion

New features should be introduced only when they provide meaningful User value.

Principle 2 — Evolution Through Validation

Changes should be based on:

- User needs,
 - observed problems,
 - real usage patterns.
-

Principle 3 — Protect Core Identity

New capabilities should strengthen Mosaica's purpose rather than change it unnecessarily.

Principle 4 — Complexity Requires Justification

Every new system component should solve a real problem.

Principle 5 — Small Controlled Evolution

Mosaica should improve through gradual iterations.

Evolution Model

Mosaica evolves through stages:

Foundation

↓

Improvement

↓

Expansion

↓

Maturity

Foundation Stage

Focus:

Building the core personal archive experience.

Includes:

- Content management,
- Library,
- Collections,
- Personal records.

Improvement Stage

Focus:

Improving existing experiences.

Examples:

- better workflows,
- usability improvements,
- organization improvements.

Expansion Stage

Focus:

Introducing additional capabilities.

Examples:

- advanced analytics,
- optional integrations,
- future AI capabilities.

Maturity Stage

Focus:

Optimizing and maintaining the ecosystem.

Future Change Evaluation

Every major change should answer:

Does it solve a real problem?

↓

Does it improve User value?

↓

Does it preserve simplicity?

↓

Is it worth the added complexity?

Module Structure

12 Product Evolution Architecture

12.01 Feature Evolution

12.02 Version Strategy

12.03 Change Management

12.04 Future Expansion

12.05 Product Evolution Review

Architectural Decisions Introduced

Decision 01

Mosaica evolves through validated needs, not assumptions.

Decision 02

Core product identity must be protected during expansion.

Decision 03

Complexity must always have justification.

Decision 04

Future capabilities are introduced gradually.

Decision 05

Evolution should preserve maintainability.

12.01 — Feature Evolution

Purpose

The purpose of this section is to define how new features are evaluated, designed and introduced into Mosaica.

This section establishes a controlled approach for product growth while preserving the original product vision.

The objective is to ensure that new features:

- provide meaningful value,
- support User goals,
- maintain system simplicity,
- avoid unnecessary complexity.

Feature Evolution Principles

Principle 1 — Problem Before Feature

Definition

New features shall begin with an identified User problem.

Application

The starting question should be:

"What problem does this solve?"

Not:

"What feature can we add?"

Example

Incorrect:

Add AI recommendations

↓

Because AI is popular

Correct:

Users struggle to discover relevant content

↓

Evaluate possible solutions

↓

Choose appropriate solution

Principle 2 — Core Value Protection

Definition

New features must strengthen Mosaica's core purpose.

Evaluation Question

Does this feature improve:

- personal archive experience?
- cultural organization?
- discovery?
- reflection?

If not, it requires additional evaluation.

Principle 3 — Essential Before Optional

Definition

Required capabilities should be developed before additional enhancements.

Feature Priority Model

Priority

- └─ Core Features
 - └─ Supporting Features
 - └─ Optional Features
-

Core Features

Required for the main product experience.

Examples:

- Library management,
 - Personal records,
 - Content organization.
-

Supporting Features

Improve existing workflows.

Examples:

- better search,
 - improved navigation.
-

Optional Features

Additional experiences.

Examples:

- advanced analytics,
- AI assistance,
- integrations.

Principle 4 — Complexity Evaluation

Definition

Every feature introduces complexity and must justify its cost.

Evaluation

Before adding a feature:

User Value

+

Implementation Cost

+

Maintenance Cost

=

Feature Decision

Principle 5 — Incremental Introduction

Definition

Features should be introduced gradually.

Application

Prefer:

Basic Version

↓

Validate

↓

Improve

↓

Expand

Avoid:

Large Feature System

↓

Unknown User Value

Feature Lifecycle

A feature follows this lifecycle:

Idea

↓

Evaluation

↓

Design

↓

Implementation

↓

Validation

↓

Evolution

Feature Evaluation Criteria

Every proposed feature should be evaluated by:

User Value

Does it improve the User experience?

Product Alignment

Does it support Mosaica's purpose?

Complexity Impact

Does it introduce unnecessary burden?

Maintenance Impact

Can it be supported long-term?

Cost Impact

Does it introduce unnecessary expenses?

Feature Categories

Mosaica features are classified as:

Feature Categories

└─ Foundation

└─ Enhancement

└─ Experimental

└─ Future Capability

Foundation Features

Purpose:

Create the essential Mosaica experience.

Enhancement Features

Purpose:

Improve existing capabilities.

Experimental Features

Purpose:

Explore possibilities without affecting the core system.

Future Capability Features

Purpose:

Reserve possibilities for later implementation.

Examples:

- AI capabilities,
 - advanced integrations.
-

Feature Decision Model

Feature Idea

↓

What problem does it solve?

↓

Does it support Mosaica vision?

↓

Is complexity justified?

↓

Approve / Reject / Postpone

Feature Evolution and Cost Control

Mosaica development shall prioritize solutions that minimize unnecessary costs.

New features should consider:

- free alternatives,
- existing capabilities,
- simple implementations.

Paid services or infrastructure should only be introduced when justified.

Architectural Decisions Confirmed

Decision 01

Features begin with problems, not ideas.

Decision 02

Core product value has priority over feature quantity.

Decision 03

Feature complexity requires justification.

Decision 04

Features evolve gradually through validation.

Decision 05

Cost impact is part of feature evaluation.

12.02 — Version Strategy

Purpose

The purpose of this section is to define the versioning strategy of Mosaica.

This section establishes how product changes are categorized, tracked and communicated throughout the evolution of the system.

The objective is to maintain a clear history of development while supporting controlled growth.

Versioning Principles

Principle 1 — Versions Represent Meaningful Change

Definition

Version numbers should communicate the significance of changes.

A version increase should represent a meaningful evolution of the product.

Principle 2 — Avoid Unnecessary Version Complexity

Definition

Version management should remain simple and understandable.

Application

Mosaica should avoid excessive version fragmentation.

The goal is clarity, not bureaucracy.

Version Model

Mosaica follows a three-part version structure:

`Major.Minor.Patch`

Example:

`1.4.2`

Major Version

Format:

`x.0.0`

Meaning

Represents major product evolution.

Examples

Major changes:

- fundamental architecture changes,
 - major product direction changes,
 - significant experience transformation.
-

Example

Mosaica 1.x

↓

Mosaica 2.0

would represent a major evolution.

Minor Version

Format:

1.X.0

Meaning

Represents new capabilities that expand the product.

Examples

- new features,
 - new workflows,
 - significant improvements.
-

Example

1.2.0

↓

New Collection Features Added

Patch Version

Format:

1.2.X

Meaning

Represents smaller improvements and corrections.

Examples

- bug fixes,
- usability improvements,
- minor adjustments.

Version Lifecycle

A version follows this process:

Change Idea

↓

Evaluation

↓

Development

↓

Validation

↓

Release Version

Initial Version Strategy

Mosaica begins with:

Version 0.x

during development.

Purpose

Version 0.x represents:

- active development,
- experimentation,
- architectural formation.

First Stable Release

The first stable version:

Version 1.0

represents:

- working core product,
 - validated architecture,
 - usable personal archive system.
-

Version Decision Examples

Example 1 — Small Fix

Problem:

A button displays incorrect information.

Result:

1.0.0

↓

1.0.1

Example 2 — New Feature

Added:

New personal collection capabilities.

Result:

1.0.0

↓

1.1.0

Example 3 — Major Transformation

Changed:

Core product structure.

Result:

1.x

↓

2.0

Version Documentation

Each important release should document:

- version number,
 - release date,
 - changes,
 - important decisions.
-

Version Relationship With Architecture

Architecture changes require additional consideration.

Example:

Feature Change

↓

May affect:

Domain

Database

API

UX

↓

Evaluate Version Impact

Version Evolution Strategy

Mosaica should evolve through controlled releases:

Build

↓

Learn

↓

Improve

↓

Expand

Cost-Aware Version Growth

Version growth should not automatically introduce:

- paid services,
- infrastructure costs,
- unnecessary dependencies.

New versions should continue respecting Mosaica's simplicity principles.

Architectural Decisions Confirmed

Decision 01

Version numbers communicate the importance of change.

Decision 02

Mosaica uses a simple Major.Minor.Patch model.

Decision 03

Version 0.x represents development phase.

Decision 04

Version 1.0 represents the first stable usable product.

Decision 05

Version growth does not automatically justify increased complexity or cost.

12.03 — Change Management

Purpose

The purpose of this section is to define how changes are evaluated, documented and introduced into Mosaica.

This section establishes a controlled approach for managing changes while preserving:

- architectural consistency,
- product vision,
- system maintainability.

The objective is to ensure that changes improve Mosaica without creating unnecessary complexity.

Change Management Principles

Principle 1 — Understand Before Changing

Definition

Changes should begin with understanding the existing system and the reason for modification.

Application

Before making a change, evaluate:

- What problem exists?
 - Why is the current solution insufficient?
 - Which parts of the system are affected?
-

Principle 2 — Preserve Existing Decisions

Definition

Existing architectural decisions should be respected unless there is a strong reason to revise them.

Application

A new idea should not automatically replace an existing design.

Existing decisions should be reviewed before modification.

Principle 3 — Document Important Changes

Definition

Important architectural or product decisions shall be documented.

Application

Changes affecting:

- architecture,
- security,
- database,
- major workflows

should have an associated decision record.

Principle 4 — Evaluate Impact Before Implementation

Definition

Changes should be evaluated across the system before development begins.

Impact Areas

A change may affect:

Change Impact

- └─ Product
 - └─ Domain
 - └─ Database
 - └─ API
 - └─ Security
 - └─ UX
 - └─ Infrastructure
-

Principle 5 — Avoid Change For Change's Sake

Definition

Not every possible improvement requires implementation.

Application

A change should provide meaningful value.

Questions:

- Does it solve a real problem?
- Does it improve the User experience?
- Is the complexity justified?

Change Lifecycle

A change follows this process:

Change Idea

↓

Evaluation

↓

Decision

↓

Implementation

↓

Validation

↓

Documentation

Change Categories

Changes are classified into four categories:

Change Types

├─ Feature Change

├─ Improvement Change

├─ Technical Change

└─ Architectural Change

Feature Change

Definition

Introduces new User capabilities.

Example

Adding a new Collection feature.

Improvement Change

Definition

Improves existing functionality.

Example

Making Library navigation easier.

Technical Change

Definition

Improves internal implementation.

Example

Improving performance or code organization.

Architectural Change

Definition

Changes fundamental system structure.

Examples

- changing data model,
 - changing core boundaries,
 - introducing new architectural layers.
-

Change Evaluation Model

Before accepting a change:

Proposed Change

↓

Why is it needed?

↓

What does it affect?

↓

What is the complexity cost?

↓

Should it be implemented?

Architecture Decision Records (ADR)

Important architectural changes shall use ADR documentation.

ADR Purpose

ADR records:

- the decision,
- context,
- alternatives,
- consequences.

Example Changes Requiring ADR

- adding AI dependency,
- changing database strategy,
- introducing paid infrastructure,
- modifying core architecture.

Change Approval Levels

Changes have different evaluation requirements.

Small Changes

Examples:

- minor UI improvement,
- bug fix.

Can proceed through normal development.

Medium Changes

Examples:

- new feature,
- workflow modification.

Require impact evaluation.

Large Changes

Examples:

- architecture modification,
- product direction change.

Require formal decision documentation.

Change and Cost Control

Changes should consider financial impact.

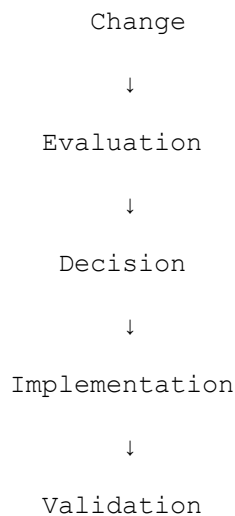
Before introducing a dependency:

Evaluate:

- Is there a free alternative?
- Is the cost necessary?
- Does the value justify the expense?

Change Management Model

Controlled Evolution



Architectural Decisions Confirmed

Decision 01

Changes require evaluation before implementation.

Decision 02

Important decisions should be documented.

Decision 03

Existing architecture should be protected.

Decision 04

Complexity and cost are part of change evaluation.

Decision 05

Large changes require formal review.

12.04 — Future Expansion

Purpose

The purpose of this section is to define possible future expansion directions for Mosaica.

This section establishes how new capabilities may be introduced while protecting the core product identity and maintaining architectural consistency.

The objective is to provide a flexible evolution path without creating premature complexity.

Future Expansion Principles

Principle 1 — Expansion Follows Need

Definition

Future capabilities shall be introduced only when real requirements emerge.

Application

A possible future feature does not automatically become a development priority.

The decision should depend on:

- User value,
- technical feasibility,
- maintenance impact.

Principle 2 — Core System Remains Independent

Definition

Future expansions should not make the core Mosaica system dependent on optional capabilities.

Application

The core archive experience should continue working without:

- AI services,
- external platforms,
- paid infrastructure,
- optional integrations.

Principle 3 — Extensions Should Be Modular

Definition

Future capabilities should be added as separate extensions when possible.

Application

New capabilities should avoid damaging existing system boundaries.

Potential Expansion Areas

Mosaica may evolve in several directions.

These are possibilities, not commitments.

1. Intelligent Assistance

Purpose

Enhance the User's cultural archive experience.

Possible Capabilities

Examples:

- personalized suggestions,
 - archive analysis,
 - discovery assistance.
-

Status

Future Capability.

Not required for initial product.

2. Advanced Organization

Purpose

Provide deeper personal archive management.

Possible Capabilities

Examples:

- advanced collections,
 - complex relationships,
 - visual organization methods.
-

Status

Possible Enhancement.

3. Analytics and Insights

Purpose

Help Users understand their own cultural patterns.

Possible Capabilities

Examples:

- consumption history,
 - personal trends,
 - preference analysis.
-

Status

Future Enhancement.

4. External Integrations

Purpose

Connect Mosaica with external ecosystems.

Possible Capabilities

Examples:

- importing information,
 - synchronization,
 - optional third-party services.
-

Status

Requires evaluation.

5. Community Features

Purpose

Introduce optional social experiences.

Possible Capabilities

Examples:

- sharing collections,
 - cultural discussions,
 - recommendations between Users.
-

Status

Requires careful evaluation.

Expansion Evaluation Model

Every future expansion should pass this evaluation:

Expansion Idea

↓

Does it strengthen Mosaica?

↓

Does it solve a real need?

↓

Can it remain modular?

↓

Is complexity justified?

↓

Consider Implementation

Expansion Risk Management

Future expansions should consider:

Product Risk

Does this change Mosaica's identity?

Technical Risk

Does this damage architectural simplicity?

Cost Risk

Does this introduce unnecessary expenses?

Maintenance Risk

Can this capability be supported long-term?

Expansion Roadmap Philosophy

Mosaica should evolve through:

Stable Foundation

↓

Useful Improvements

↓

Validated Expansion

↓

Advanced Capabilities

Future Expansion Boundaries

Mosaica should avoid becoming:

- a general social network without purpose,
 - an unnecessary data collection platform,
 - a technology showcase without User value.
-

Architectural Decisions Confirmed

Decision 01

Future capabilities are possibilities, not commitments.

Decision 02

The core archive experience remains independent.

Decision 03

Expansion requires validation.

Decision 04

New capabilities should preserve modularity.

Decision 05

Cost and complexity are part of expansion decisions.

12.05 — Product Evolution Review

Review Objective

The purpose of this Architecture Review is to validate that the Mosaica Product Evolution Architecture provides a controlled and sustainable approach for future growth.

This review confirms that product evolution decisions preserve:

- the original product vision,
 - architectural consistency,
 - simplicity,
 - long-term maintainability.
-

Scope Reviewed

The following product evolution components have been reviewed:

- Feature Evolution
 - Version Strategy
 - Change Management
 - Future Expansion
-

Validation Results

Product Vision Protection

Status: Approved

The Product Evolution Architecture protects the core identity of Mosaica.

Confirmed:

Mosaica remains focused on:

- personal cultural archive,
- organization of knowledge,
- meaningful User relationships with Content.

Future additions must strengthen this purpose.

Feature Evolution Strategy

Status: Approved

Feature growth follows a problem-first approach.

Confirmed:

Problem

↓

User Value

↓

Solution

↓

Implementation

Features are not introduced only because they are technically possible.

Version Strategy

Status: Approved

Version management provides a clear method for tracking product evolution.

Confirmed:

Major

Minor

Patch

structure is sufficient for Mosaica's needs.

Change Management

Status: Approved

Changes are evaluated before implementation.

Confirmed:

Important changes consider:

- product impact,
 - architectural impact,
 - cost impact,
 - maintenance impact.
-

Future Expansion Strategy

Status: Approved

Future capabilities remain optional and controlled.

Confirmed:

Future areas such as:

- AI capabilities,
- advanced analytics,
- integrations,
- community features

are not required for the initial product.

Complexity Control

Status: Approved

Mosaica follows controlled growth principles.

Confirmed:

The system should avoid:

- unnecessary features,
- unnecessary dependencies,
- premature investments.

Cost Management

Status: Approved

Product evolution decisions include financial considerations.

Confirmed:

New capabilities should evaluate:

- free alternatives,
- operational cost,
- actual value.

Paid services should only be introduced when justified.

Evolution Maturity Model

Mosaica evolution follows:

Foundation

↓

Improvement

↓

Expansion

↓

Maturity

The system should not skip directly to advanced stages without a stable foundation.

Architectural Decisions Confirmed

Decision 01

Mosaica grows through validated needs.

Decision 02

The core product identity must be protected.

Decision 03

Feature quantity does not define product quality.

Decision 04

Complexity and cost must be justified.

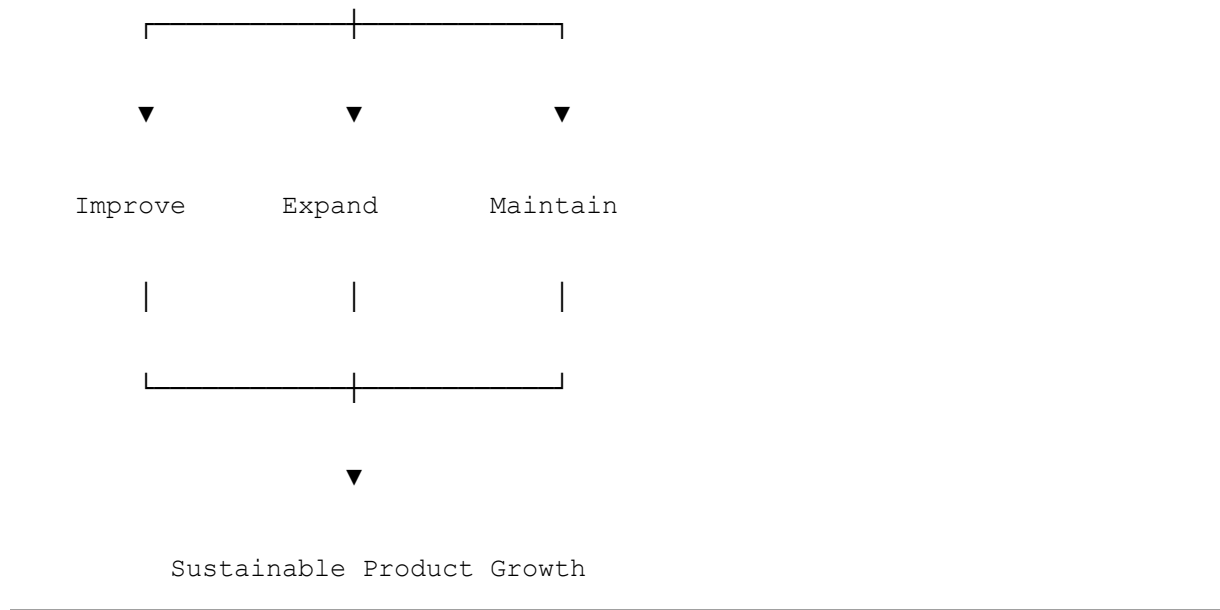
Decision 05

Future capabilities remain optional until proven valuable.

Product Evolution Summary

Product Evolution

|



Review Conclusion

The Mosaica Product Evolution Architecture is considered complete.

The current design provides:

- controlled feature growth,
- protection of the original vision,
- sustainable future expansion,
- responsible complexity management.

No blocking product evolution issues have been identified.